

Assignment 01 — Observability in the Wild

Architecture Analysis

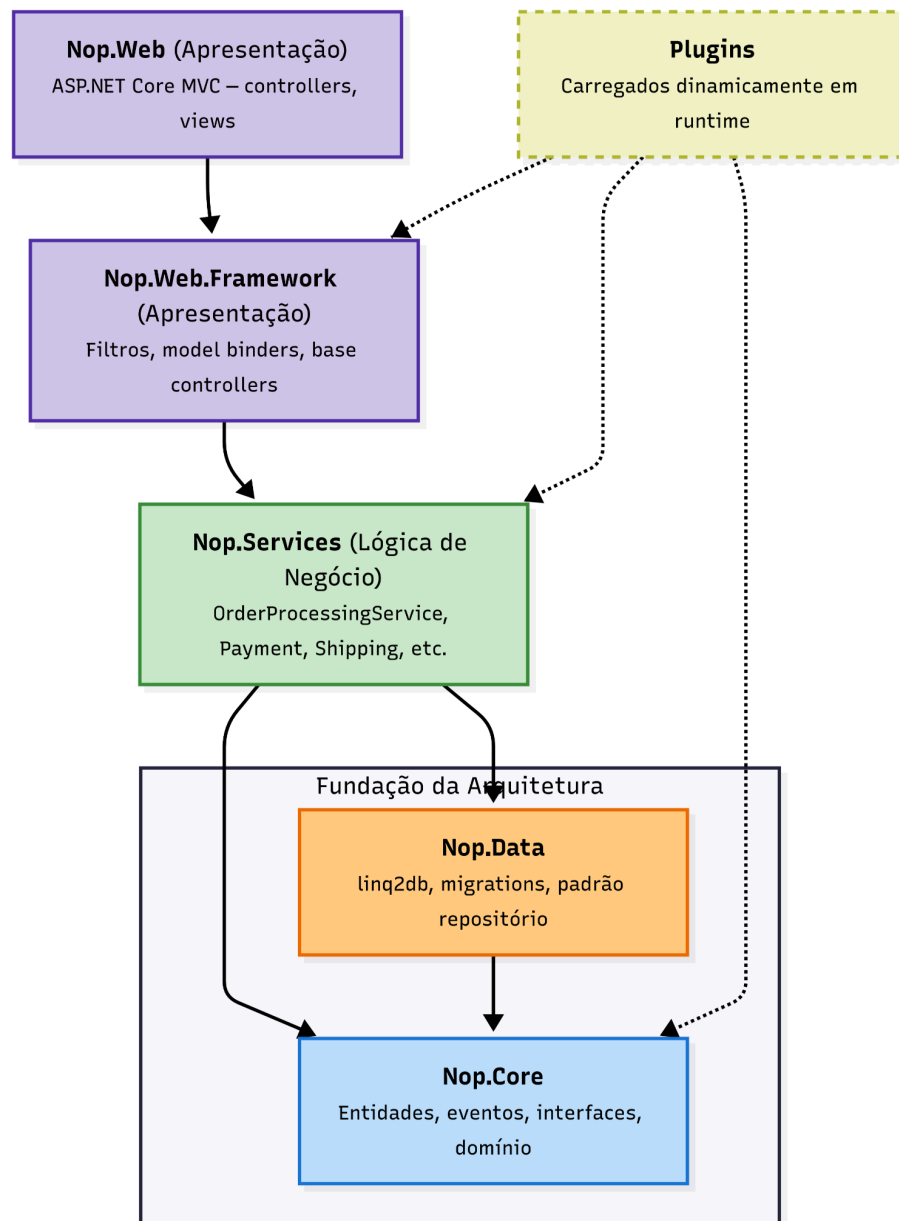
Guilherme Silva, Nmec 131143

Março 2026

Task 1 - Read Before You Touch – Architecture Analysis

1. How are the layers organised and what are the dependency rules between them?

Examining the project's .csproj files, I identified a strict layered architecture with four main projects and a plugin system:



This diagram illustrates the layered architecture of nopCommerce. The main rule is that dependencies always flow from top to bottom (the arrows indicate what depends on what):

- **The Base (Nop.Core):** It is the heart of the system. It contains the domain entities and rules, and it does not depend on any other project.
- **The Engine (Nop.Data and Nop.Services):** *Data* connects to the database, while *Services* executes the business logic (processing orders, payments, etc.).
- **The Interface (Nop.Web and Framework):** This is the website itself (*ASP.NET Core*). It depends on the underlying layers to retrieve information and display it to the user.
- **The Plugins:** These are independent (dashed lines). They are loaded separately and connect to the main system to add functionalities without the need to alter the original code.

The dependency rules, confirmed by the project references, are as follows:

Project	References
Nop.Core	No internal projects (absolute domain base).
Nop.Data	Only Nop.Core.
Nop.Services	Nop.Core + Nop.Data
Nop.Web.Framework	Nop.Core + Nop.Data + Nop.Services
Nop.Web	All of the above projects.

Plugins	Typically Nop.Core + Nop.Services + Nop.Web.Framework
----------------	--

The rule is strictly unidirectional — no lower layer references an upper one.

Nop.Core knows nothing about HTTP, services, or the database. This separation is enforced by MSBuild project references, not just by convention.

The plugin system is loaded at runtime, operating in a modular fashion outside the main dependency graph. Plugins implement interfaces defined in **Nop.Core** or **Nop.Services** and are resolved via **Autofac** (which remains the default Dependency Injection container and is recommended by the team to handle modular complexity), although the architecture allows switching to the native .NET container via system settings.

2. How does nopCommerce handle events internally — what is IEventPublisher and how is it used?

What is it?

The `IEventPublisher` is the internal pub/sub (publish/subscribe) system of nopCommerce. It is defined in `Nop.Core/Events/IEventPublisher.cs` with a single method:

```
1 namespace Nop.Core.Events;
2
3 /// <summary>
4 /// Represents an event publisher
5 /// </summary>
6 public partial interface IEventPublisher
7 {
8     /// <summary>
9     /// Publish event to consumers
10    /// </summary>
11    /// <typeparam name="TEvent">Type of event</typeparam>
12    /// <param name="event">Event object</param>
13    /// <returns>A task that represents the asynchronous operation</returns>
14    Task PublishAsync<TEvent>(TEvent @event);
15 }
```

The concrete implementation (`Nop.Services/Events/EventPublisher.cs`) resolves all `IConsumer<TEvent>` instances registered in the DI (Dependency Injection) container and invokes `HandleEventAsync` on each one:

```
public virtual async Task PublishAsync<TEvent>(TEvent @event)
{
    //get all event consumers
    var consumers = EngineContext.Current.ResolveAll<IConsumer<TEvent>>().ToList();

    foreach (var consumer in consumers)
    {
        try
        {
            //try to handle published event
            await consumer.HandleEventAsync(@event);

            if (@event is IStopProcessingEvent { StopProcessing: true })
                break;
        }
    }
}
```

How it is used

Services receive `IEventPublisher` via constructor injection and trigger events during the most important business transitions. When analyzing the `OrderProcessingService`, I identified the following trigger points:

Event	When it is triggered
<code>OrderPlacedEvent</code>	Order successfully placed
<code>OrderPaidEvent</code>	Payment confirmed
<code>OrderStatusChangedEvent</code>	Any state transition
<code>OrderRefundedEvent</code>	Full or partial refund
<code>OrderVoidedEvent</code>	Payment voided
<code>EntityInsertedEvent<T></code>	Any entity inserted into the repository

There are also extension methods (`EventPublisherExtensions`) for entity lifecycle events: `EntityInserted`, `EntityUpdated`, `EntityDeleted`.

Critical problem for observability

Upon examining the `EventPublisher` implementation, I found that it uses a service locator (`EngineContext.Current.ResolveAll<>`) instead of constructor-injected dependencies. This has two direct consequences:

- It is not possible to intercept or decorate event dispatching with a standard decorator — any extension requires modifying the `EventPublisher` directly.
- The tracing context (`Activity`) is not automatically propagated through the dispatch. Consumers inherit `Activity.Current` only by accident from synchronous execution on the same thread.

Additionally, consumer exceptions are silenced: the `EventPublisher` logs the error but does not propagate it, which means a failing OTel (OpenTelemetry) consumer would fail silently.

3. Where does the code make it easy to add observability, and where does it make it hard?

What makes it easier

- **Rich domain events already exist:** `OrderPlacedEvent`, `OrderPaidEvent`, etc., are triggered exactly at the business boundaries relevant for tracing. It is possible to create an `IConsumer<OrderPlacedEvent>` to record a span without touching the `OrderProcessingService`.
- **Interface-oriented services:** All services sit behind an interface, which makes direct decoration possible without modifying the original code.
- **Native integration with ASP.NET Core:** Using `AddOpenTelemetry()` with `AddAspNetCoreInstrumentation()` provides HTTP-level tracing without any code changes in the controllers.
- **Constructor injection everywhere:** Services receive their dependencies via constructor, so an `ActivitySource` or `IMeterFactory` can be injected into a new consumer without performing surgery on the existing code.

What makes it harder

- **Big classes:** `OrderProcessingService.cs` has 3531 lines, and `CheckoutController.cs` has 2185 lines. Identifying the correct instrumentation point requires careful reading; there are no obvious seams.
- **Service locator in the EventPublisher:** Because `EventPublisher` uses `EngineContext.Current`, it is not possible to intercept event dispatching with standard middleware or decorators.
- **Lack of trace context in event objects:** Domain event classes (e.g., `OrderPlacedEvent`) carry only the entity — no correlation ID, no trace context. Linking a consumer span to its parent depends on `Activity.Current` being correctly set at the time of the call.
- **Sensitive data in the Order entity:** The entity exposed by `OrderPlacedEvent` contains `CardNumber`, `CardName`, `CardCvv2`, `CardExpirationMonth`, `CardExpirationYear`, `CustomerIp`, and `VatNumber`. Any span attribute derived from this object must be sanitized before being exported. There is no built-in sanitization layer.
- **Silent consumer failures:** If an OTel (OpenTelemetry) consumer throws an exception, the error is logged, but the business flow continues. Span recording failures would be invisible without dedicated monitoring of the consumer itself.

4. What would you need to change structurally to instrument it properly — and is that change worth making?

Required Changes

I have identified three minimal additions sufficient to instrument the "Customer places an order" flow end-to-end:

1. **Change 1 — Register OpenTelemetry in `Program.cs` (no existing code modified):** Add `AddOpenTelemetry()` to the service collection with `AddAspNetCoreInstrumentation()`. This provides HTTP and database-level spans automatically.
2. **Change 2 — Create an `IConsumer<T>` for OTel (new file, no existing code modified):** Create a class, for example, `OrderTelemetryConsumer`, that implements `IConsumer<OrderPlacedEvent>` and `IConsumer<OrderPaidEvent>`. Each handler starts and stops an `Activity` with safe and sanitized attributes. The framework's startup infrastructure (`NopStartup.cs`) discovers and registers all `IConsumer<T>` implementations automatically via `ITypeFinder`.
3. **Change 3 — A surgical edit to `EventPublisher.cs` (minimal and justified):** Add an `ActivitySource` to `EventPublisher` so that the event dispatching itself becomes an observable span. It is a 5-line change in a single file and does not modify any business logic.

Is it worth making these changes?

Yes, with clear boundaries. The edit to the `EventPublisher` is low-risk (it adds instrumentation, it does not alter logic) and high-value (it covers the entire internal event bus, not just today's target events). The already existing `IConsumer<T>` mechanism is sufficient to instrument the order flow without destabilizing the codebase.

What would be worth changing in a production context would be:

- **Propagating a `TraceId`** within the domain event objects to link consumer spans even in asynchronous contexts.
- **Adding a dedicated OTel Processor** to sanitize PII (Personally Identifiable Information) before exporting the spans.

These changes would imply modifying the event classes in `Nop.Core` and the `IEventPublisher` interface — a larger impact that is not justified for point-in-time instrumentation, but which would structurally improve the system's observability.